Using Namespaces

It takes only a short while for you to get a chance to apply your newfound knowledge about classes. Sifting through the Bookworm source code, you stumble upon a class that represents a snippet of text out of a book:

```
class TEXT
  # ...
```

Ruby class names are conventionally Pascal cased: words are concatenated with the first letter of each capitalized: ThisTextIsPascalCased, so you rename the class Text:

```
class Text
```

You change the name of the class everywhere it's used, you run the unit tests, and—surprise!—the tests fail with a cryptic error message:

```
TypeError: Text is not a class
```

"D'oh! Of course it is," you exclaim. Bill is as puzzled as you are, so it takes the two of you some time to find the cause of the problem. As it turns out, the Bookworm application requires an old version of the popular Action Mailer library. Action Mailer, in turn, uses a text-formatting library that defines a module named—you guessed it—Text:

```
module Text
```

That's where the problem lies: because Text is already the name of a module, Ruby complains that it can't also be the name of a class at the same time.

In a sense, you were lucky that this name clash was readily apparent. If Action Mailer's Text had been a class, you might have never noticed that this name already existed. Instead, you'd have inadvertently *Monkeypatched (16)* the existing Text class. At that point, only your unit tests would have protected you from potential bugs.

Fixing the clash between your Text class and Action Mailer's Text module is as easy as wrapping your class in a *Namespace (23)*:

```
module Bookworm
  class Text
```

You and Bill also change all references to Text into references to Bookworm::Text. It's unlikely that an external library defines a class named Bookworm::Text, so you should be safe from clashes now.

That was a lot of learning in a single sitting. You deserve a break and a cup of coffee—and a little quiz.